

Enterprise Architecture Notes: Advanced Mocking & CI/CD Resilience

Sentinel Project Notebook

May 8, 2026

1 Framework Magic vs. Pure Java (Defeating the Ghost)

In complex Spring WebFlux applications, relying on annotations like `@Mock`, `@InjectMocks`, or `@SpringBootTest` hands control to the Spring and JUnit lifecycles.

The Problem: Spring may attempt to build a partial application context, intercepting the annotations and injecting real, un-stubbed beans or proxies instead of your intended mock. This leads to untraceable `NullPointerExceptions`.

The Nuclear Option (Pure Java): Bypass the framework entirely by manually instantiating mocks and injecting them via constructor. This guarantees 100% isolation in JVM memory.

```
1 // 1. Pure Java Mock Generation
2 ConversationalMemoryService memoryService = Mockito.mock(
    ConversationalMemoryService.class);
3
4 // 2. Manual Constructor Injection
5 AnalysisService analysisService = new AnalysisService(chatModel, memoryService)
    ;
```

2 Behavioral Verification (The SRE Tripwire)

A test that requires human eyeballs to verify logs is technical debt. Instead of just verifying that a method didn't crash, we must mathematically prove that the correct inputs (like Tenant IDs) were passed through the system.

Mockito Verification API:

```
1 Mockito.verify(memoryService).getHistory(
2     Mockito.eq("EXPECTED_TENANT"),
3     Mockito.eq("TEST_SESSION")
4 );
```

This acts as a tripwire in the CI/CD pipeline. If the routing logic accidentally alters or drops the Tenant ID, the pipeline will instantly fail, ensuring perfect architectural isolation verification.

3 Asynchronous Verification in Project Reactor

In reactive programming, background tasks (like `.doOnComplete()`) execute on separate threads. Standard assertions will fail because the main test thread evaluates before the background thread finishes.

The Timeout Strategy: Use `Mockito.timeout()` to tell the test thread to wait and poll for the asynchronous execution.

```
1 Mockito.verify(memoryService, Mockito.timeout(500)).saveInteraction(  
2     Mockito.eq("EXPECTED_TENANT"),  
3     Mockito.eq("TEST_SESSION"),  
4     Mockito.anyString(),  
5     Mockito.anyString()  
6 );
```

4 Under the Hood: The Imposter and the Ledger

When `Mockito.mock(Class.class)` is called, Mockito uses bytecode manipulation (ByteBuddy) to generate a dynamic proxy—an *imposter*.

- **Logic Deletion:** All internal business logic (e.g., database calls) is stripped out.
- **The Invocation Ledger:** The imposter records every method call and its arguments into an in-memory list (the ledger).
- **The Audit:** When `Mockito.verify()` is called, it interrogates this ledger to confirm the exact sequence and parameters of the calls. The database is never touched, ensuring hyper-fast unit testing.